

EVIDENCIAS ACADÉMICAS · SEMANA 4 · ACTIVIDAD 2

Serialización y persistencia con JSON

Quantum Core — Del objeto en memoria al archivo en disco

Evidencia técnica basada en el código y la ejecución real del proyecto

ESTUDIANTE	Deibis Zuluaga Baena
NÚCLEO	Fundamentos de Software
ACTIVIDAD	Semana 4 · Actividad 2 — Serialización y persistencia (JSON)
PROYECTO	Quantum Core
ARCHIVO PRINCIPAL	serializacion_json.py
DATOS DE PRUEBA	10 transacciones simuladas de los cuatro tipos
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
FECHA DE ENTREGA	13 de septiembre de 2026

Objetivo y criterios de evaluación

El documento explica y demuestra el proceso de serialización y deserialización de objetos POO a JSON, y su aplicación a un caso concreto de persistencia.

Objetivo. Comprender y explicar el proceso de serialización y deserialización de objetos POO a formato JSON, demostrando cómo preparar los datos para el almacenamiento o la transferencia entre sistemas. La evidencia se basa en el código y la ejecución real del proyecto ubicado en `ProyectosCeipa/Semana_4_Act_2`.

Dónde se atiende cada criterio de la rúbrica

CRITERIO	DÓNDE SE ATIENDE	EVIDENCIA
Comprensión de la serialización y deserialización	Sección 02 — fundamentos, analogía del viaje y por qué JSON	Páginas 3 y 4
Implementación del proceso	Sección 03 — el código de ambos sentidos, con las etapas separadas	Figuras 1 a 3
Manejo de casos de error	Sección 05 — siete orígenes de fallo y las decisiones de diseño	Figuras 7 y 8
Documentación y explicación del proceso	Todo el documento y el <code>README.md</code> de la carpeta	Páginas 3 a 12
Aplicación a un problema concreto	Sección 04 — las 10 transacciones persistidas en disco	Figuras 4 a 6

Archivos que componen la entrega

ARCHIVO	CONTENIDO
<code>serializacion_json.py</code>	Clases, fábrica de objetos, serialización, deserialización y persistencia
<code>transacciones.txt</code>	10 transacciones simuladas de los cuatro tipos, en formato <code>cliente_id, tipo, monto</code>
<code>transacciones.json</code>	Archivo serializado que genera el programa en cada ejecución
<code>README.md</code>	Documentación del desarrollo, los resultados y las decisiones tomadas

EL RECORRIDO COMPLETO

`transacciones.txt` → objetos Python → `transacciones.json`

Qué es serializar y por qué es obligatorio

Paso 1 del enunciado: la analogía del viaje y la razón por la que un objeto no se puede guardar ni enviar tal como está.

La analogía del viaje

Serializar es **empacar**: no se puede enviar un armario por correo, así que su contenido se mete en una caja plana y etiquetada. El objeto se convierte en texto, y el texto sí cabe en un archivo o en una petición HTTP. Deserializar es **desempacar** al llegar: se abre la caja y se vuelve a armar el objeto, con todo su comportamiento.

Por qué la conversión es obligatoria

Los objetos viven en la memoria del programa. Existen mientras este corre y desaparecen cuando termina, porque el espacio de memoria que ocupaban se libera. Un intento de guardarlos directamente falla:

```
INTENTO DIRECTO: json.dumps(transaccion)
```

```
TypeError: Object of type TransaccionCredito is not JSON serializable
```

JSON solo entiende textos, números, listas y diccionarios. Una clase creada dentro del proyecto no está en esa lista, y el módulo `json` no tiene forma de saber qué hacer con ella. De ahí la escala intermedia obligatoria.

El viaje completo, en dos escalas

SENTIDO	RECORRIDO	HERRAMIENTA
Serializar	Objeto → Diccionario → Texto JSON	<code>json.dumps()</code>
Deserializar	Texto JSON → Diccionario → Objeto	<code>json.loads()</code> + constructor

La escala del medio no es opcional: el módulo `json` no sabe traducir un objeto de una clase propia, pero sí sabe traducir un diccionario. Conviene retener un detalle que se confunde con frecuencia: `json.dumps()` no devuelve un diccionario, devuelve un `str`; y `json.loads()` no devuelve un objeto, devuelve un diccionario. El último tramo hasta el objeto es siempre manual.

Por qué JSON y no un texto plano cualquiera

La pregunta es legítima: si JSON también es texto, ¿por qué no usar el formato plano de la Actividad 1, `C001,CREDITO,500000`? Los dos son texto; la diferencia está en lo que cada formato **conserva**.

ASPECTO	TEXTO PLANO	JSON
Tipos de dato	Todo vuelve como texto: <code>'500000'</code> es <code>str</code>	El número vuelve como número: <code>500000</code> es <code>float</code>
Significado	Hay que saber de antemano que el tercer campo es el monto	El nombre viaja pegado al valor: el dato se explica solo
Errores de formato	Una columna faltante pasa inadvertida	Lanza <code>JSONDecodeError</code> indicando línea y columna
Estructura	Plano: una fila de campos sueltos	Admite objetos anidados y listas dentro de listas
Compatibilidad	Exige escribir un lector propio en cada sistema	<code>json.loads()</code> está en la librería estándar de casi todos los lenguajes

La consecuencia concreta en este proyecto

La primera fila de esa tabla no es teórica. Como el texto plano devuelve el monto en forma de cadena, hay que convertirlo a mano con `float()`, y **esa conversión es exactamente la que produjo los `ValueError` de la Actividad 1** al encontrarse con `texto_invalido`. JSON elimina ese paso porque el tipo de dato viaja dentro del formato, no se deduce después.

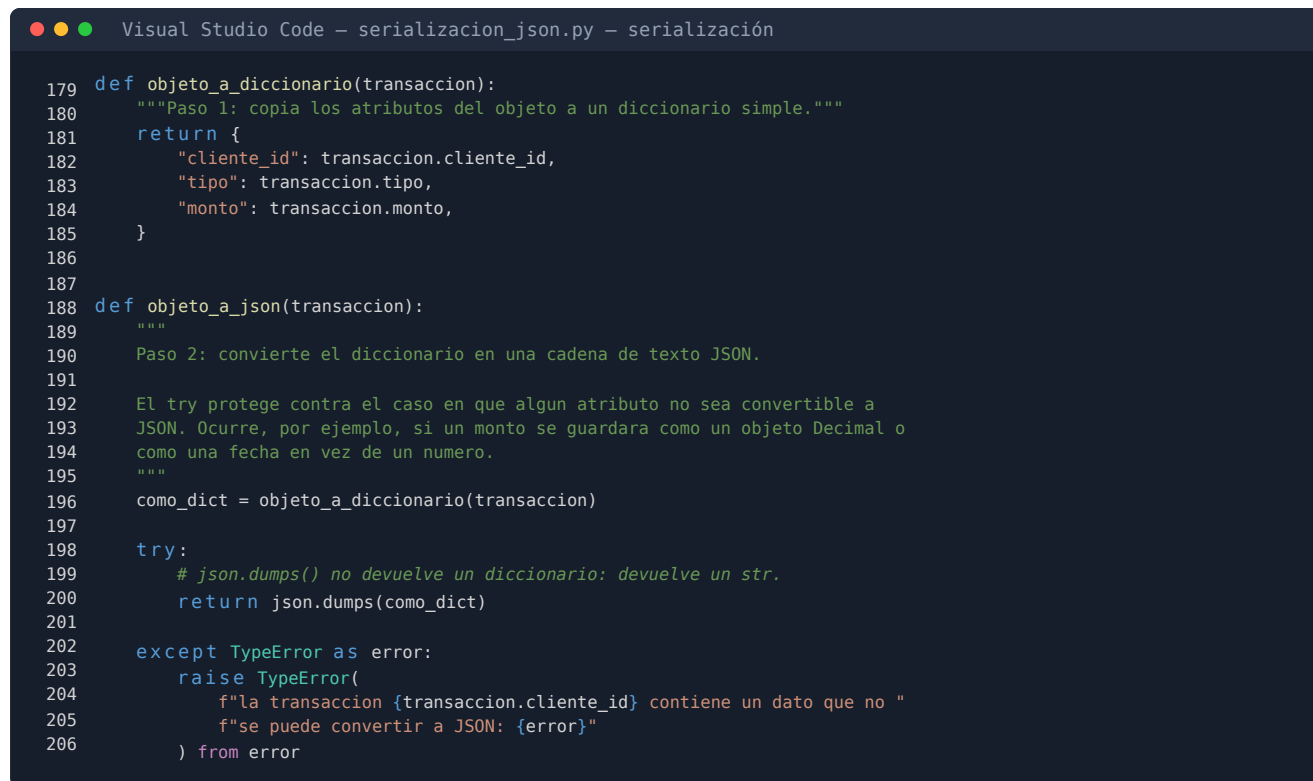
Sobre la compatibilidad

Un formato propio obliga a escribir y mantener un lector en cada sistema que quiera consumir los datos, y a mantenerlos sincronizados cuando el formato cambie. JSON traslada ese trabajo a una librería que ya existe, probada y mantenida por terceros. Esa es la razón por la que se convirtió en el idioma estándar de las APIs, incluida la del Proyecto Completo del curso, donde el *frontend* consume con `fetch()` exactamente el mismo formato que aquí se produce.

Serialización: Objeto → Diccionario → JSON

Paso 2 del enunciado, primer sentido. Las dos etapas se dejaron en funciones separadas para que la secuencia quede visible.

Separar `objeto_a_diccionario()` de `objeto_a_json()` es una decisión deliberada. Podrían ser una sola función, pero entonces la escala intermedia quedaría escondida dentro de una llamada, y es precisamente la etapa que explica por qué el proceso necesita dos pasos y no uno.



```
Visual Studio Code – serializacion_json.py – serialización

179 def objeto_a_diccionario(transaccion):
180     """Paso 1: copia los atributos del objeto a un diccionario simple."""
181     return {
182         "cliente_id": transaccion.cliente_id,
183         "tipo": transaccion.tipo,
184         "monto": transaccion.monto,
185     }
186
187
188 def objeto_a_json(transaccion):
189     """
190     Paso 2: convierte el diccionario en una cadena de texto JSON.
191
192     El try protege contra el caso en que algun atributo no sea convertible a
193     JSON. Ocurre, por ejemplo, si un monto se guardara como un objeto Decimal o
194     como una fecha en vez de un numero.
195     """
196     como_dict = objeto_a_diccionario(transaccion)
197
198     try:
199         # json.dumps() no devuelve un diccionario: devuelve un str.
200         return json.dumps(como_dict)
201
202     except TypeError as error:
203         raise TypeError(
204             f"la transaccion {transaccion.cliente_id} contiene un dato que no "
205             f"se puede convertir a JSON: {error}"
206         ) from error
```

Figura 1. Etapa 1: los atributos se copian a un diccionario. Etapa 2: `json.dumps()` lo convierte en texto.

Deserialización: JSON → Diccionario → Objeto

```

Visual Studio Code – serializacion_json.py – deserialización

212 CLAVES_OBLIGATORIAS = ("cliente_id", "tipo", "monto")
213
214 def diccionario_a_objeto(como_dict):
215     """
216     Reconstruye el objeto a partir del diccionario, validando antes sus claves.
217
218     Se revisan las claves en lugar de dejar que Python lance un KeyError seco.
219     Un KeyError solo dice 'monto'; este mensaje dice que registro venia
220     incompleto y que le falta, que es lo que necesita quien corrige los datos.
221     """
222     faltantes = [clave for clave in CLAVES_OBLIGATORIAS if clave not in como_dict]
223
224     if faltantes:
225         raise ValueError(f"al registro le faltan las claves: {'', '.join(faltantes)}")
226
227     # La fabrica lanza ValueError si el tipo no existe, y el constructor lo
228     # lanza si el monto no es un numero. Ambos suben tal cual a quien llama.
229     return crear_transaccion(
230         como_dict["cliente_id"],
231         como_dict["tipo"],
232         como_dict["monto"],
233     )
234
235
236
237 def json_a_objeto(texto_json):
238     """
239     Camino inverso: del texto al diccionario, y del diccionario al objeto.
240
241     Un texto mal formado hace fallar a json.loads() con JSONDecodeError. Ese
242     error se traduce a ValueError para que quien llame maneje un solo tipo de
243     fallo de datos, sin tener que conocer el detalle interno del modulo json.
244     """
245     # Paso 1: json.loads() solo llega hasta el diccionario.
246     try:
247         como_dict = json.loads(texto_json)
248
249     except json.JSONDecodeError as error:
250         raise ValueError(f"el texto recibido no es un JSON valido: {error}") from error
251
252     # Paso 2: con sus claves se llama al constructor. Ahi vuelven los metodos.
253     return diccionario_a_objeto(como_dict)

```

Figura 2. El último tramo, del diccionario al objeto, es manual: se llama al constructor con las claves.

La función `diccionario_a_objeto()` no llama directamente al constructor de una clase concreta, sino a la fábrica `crear_transaccion()`. La razón se explica en la página siguiente y es uno de los hallazgos de esta actividad.

Evidencia: el viaje de ida y vuelta

```
Terminal — python3 serializacion_json.py

--- Viaje de ida y vuelta de una transaccion ---

1) Objeto original : Transaccion [CREDITO ] - ID: C001 | Monto: $ 500,000.00
   Clase de Python : TransaccionCredito
   Impacto         : $10,000.00

2) Diccionario     : {'cliente_id': 'C001', 'tipo': 'CREDITO', 'monto': 500000.0}
   Tipo de dato     : dict

3) Texto JSON      : {"cliente_id": "C001", "tipo": "CREDITO", "monto": 500000.0}
   Tipo de dato     : str (ya puede viajar o guardarse)

4) Objeto reconstruido : Transaccion [CREDITO ] - ID: C001 | Monto: $ 500,000.00
   Clase de Python   : TransaccionCredito
   Impacto           : $10,000.00 (el metodo volvio)
```

Figura 3. Las cuatro etapas del recorrido, con el tipo de dato de cada una.

Qué se pierde y qué se recupera

Se pierden los métodos. En el texto JSON solo quedan los datos. El método `calcular_impacto()` no aparece por ninguna parte. Vuelve al deserializar, porque se lo devuelve la clase en el momento de llamar al constructor.

No viaja la clase. El JSON tampoco guarda que el objeto era un `TransaccionCredito`: lo único que viaja es el dato `"tipo": "CREDITO"`. Es la fábrica la que lee ese texto en el destino y decide qué clase construir. Por eso la deserialización pasa por `crear_transaccion()` y no por el constructor de una clase fija: en el momento de reconstruir todavía no se sabe cuál corresponde.

No es el mismo objeto. El resultado es una copia nueva con los mismos datos, en otra dirección de memoria. El original se queda donde estaba.

Del texto en memoria al archivo en disco

Aplicación a un problema concreto: las diez transacciones simuladas se guardan y se recuperan de un archivo real.

Persistir significa permanecer. Los datos persistentes son los que siguen existiendo después de que el programa termina. La memoria es volátil; el disco es persistente. Conviene distinguir los dos términos del título de la actividad: la **serialización** es la técnica —traducir el objeto a texto—, y la **persistencia** es el resultado —que ese texto quede guardado—. Se puede serializar sin persistir, que es lo que ocurre cuando un JSON viaja por la red y nadie lo guarda; pero no se puede persistir sin serializar, porque un disco no almacena objetos de Python, solo texto y bytes.

Para escribir en disco se usan `json.dump()` y `json.load()`, sin la **s** final. Las versiones con **s** trabajan con texto en memoria; estas dos hacen la misma traducción y además leen o escriben el archivo.



```
Visual Studio Code – serializacion_json.py – guardar_transacciones()

262 def guardar_transacciones(lista_transacciones, nombre_archivo):
263     """
264     Serializa la lista completa de objetos y la escribe en un archivo .json.
265
266     Un fallo de escritura (permisos, disco lleno, ruta inexistente) llega como
267     OSError y se reporta con el nombre del archivo, que es el dato útil.
268     """
269     lista_de_diccionarios = [objeto_a_diccionario(t) for t in lista_transacciones]
270
271     try:
272         with open(nombre_archivo, "w", encoding="utf-8") as archivo:
273             json.dump(lista_de_diccionarios, archivo, indent=2, ensure_ascii=False)
274
275     except OSError as error:
276         raise OSError(f"no se pudo escribir {nombre_archivo}: {error}") from error
277
278     return len(lista_de_diccionarios)
```

Figura 4. Cada objeto se convierte en diccionario antes de escribir: `json.dump()` no maneja objetos.

Recuperar los objetos desde el archivo

```

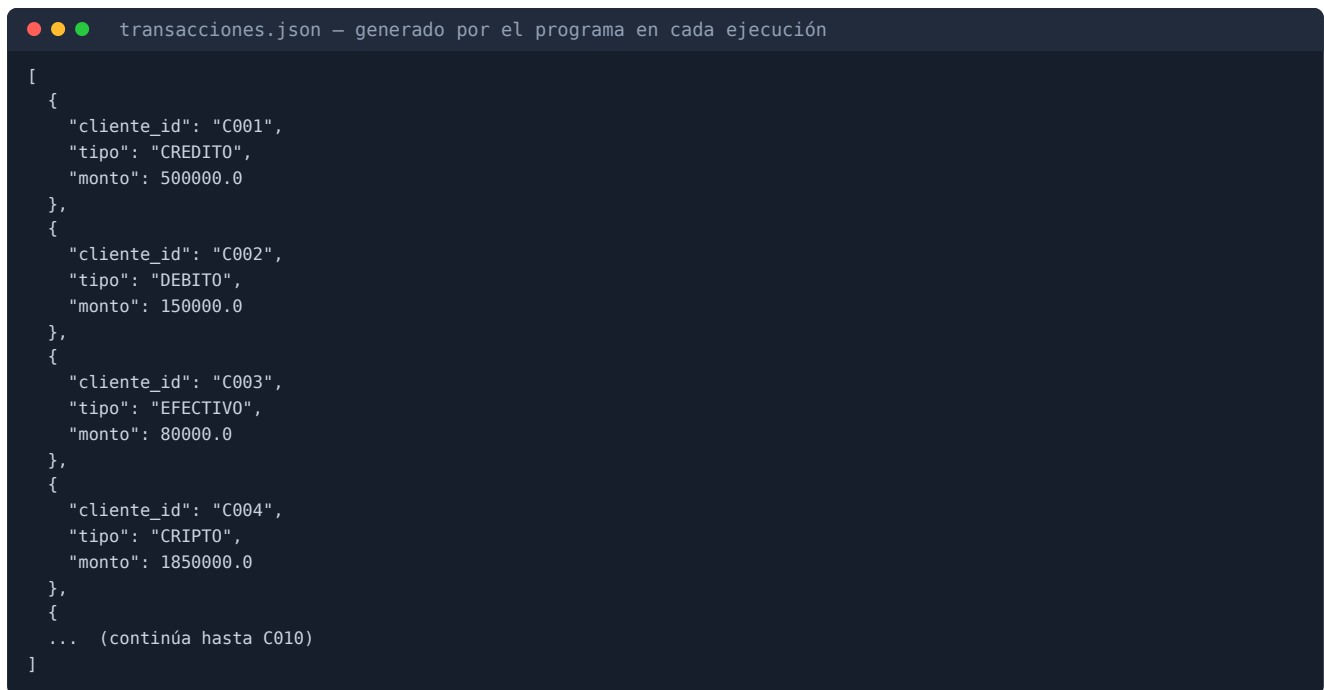
Visual Studio Code – serializacion_json.py – cargar_transacciones()

281 def cargar_transacciones(nombre_archivo):
282     """
283     Lee el archivo .json y reconstruye la lista de objetos con sus metodos.
284
285     Se distinguen dos clases de fallo, y se tratan distinto a proposito:
286
287     - Los que impiden continuar (el archivo no existe, o su contenido no es
288       JSON) detienen la carga, porque no hay nada que recuperar.
289     - Los de un registro suelto (le falta una clave, el tipo no existe, el
290       monto no es numero) se aíslan: se deja constancia y se sigue con el
291       siguiente, igual que en la Actividad 1.
292     """
293     try:
294         with open(nombre_archivo, "r", encoding="utf-8") as archivo:
295             lista_de_diccionarios = json.load(archivo)
296
297     except FileNotFoundError as error:
298         raise FileNotFoundError(
299             f"no existe el archivo {nombre_archivo}; se genera al ejecutar "
300             f"guardar_transacciones()"
301         ) from error
302
303     except json.JSONDecodeError as error:
304         raise ValueError(
305             f"{nombre_archivo} existe pero su contenido no es un JSON valido: {error}"
306         ) from error
307
308     transacciones = []
309
310     for numero, registro in enumerate(lista_de_diccionarios, start=1):
311         try:
312             transacciones.append(diccionario_a_objeto(registro))
313
314         except ValueError as error:
315             print(f"    [Registro {numero}] descartado -> {error}")
316
317     return transacciones

```

Figura 5. Dos clases de fallo tratadas distinto: las que impiden continuar detienen la carga, las de un registro se aíslan.

El archivo generado



```
transacciones.json — generado por el programa en cada ejecución

[
  {
    "cliente_id": "C001",
    "tipo": "CREDITO",
    "monto": 500000.0
  },
  {
    "cliente_id": "C002",
    "tipo": "DEBITO",
    "monto": 150000.0
  },
  {
    "cliente_id": "C003",
    "tipo": "EFECTIVO",
    "monto": 80000.0
  },
  {
    "cliente_id": "C004",
    "tipo": "CRIPTO",
    "monto": 1850000.0
  },
  {
    ... (continúa hasta C010)
  }
]
```

Figura 6. El archivo queda legible tanto para una persona como para cualquier otro sistema.

La prueba de que la persistencia funciona es que un programa distinto, que no crea ninguna transacción, puede abrir ese archivo y encontrar las diez. Los datos sobrevivieron al programa que los creó, algo que la memoria no permite.

Un archivo `.json` es la forma más simple de persistir. Las bases de datos, que llegan más adelante en el curso, hacen lo mismo y además permiten buscar, filtrar y que varios procesos escriban a la vez sin pisarse, pero el concepto de fondo es el mismo que se aplica aquí.

Los fallos previstos y cómo se tratan

Traducir datos entre dos mundos es donde más cosas pueden fallar. La estrategia es la misma de la Actividad 1: detectar, dejar constancia y continuar.

ORIGEN DEL FALLO	EXCEPCIÓN	DÓNDE SE DETECTA
Serializar un objeto con un dato no convertible	<code>TypeError</code>	<code>objeto_a_json()</code>
Texto JSON mal formado	<code>JSONDecodeError</code>	<code>json_a_objeto()</code>
Falta una clave en el registro	<code>ValueError</code>	<code>diccionario_a_objeto()</code>
Tipo de transacción desconocido	<code>ValueError</code>	<code>crear_transaccion()</code>
Monto no numérico o negativo	<code>ValueError</code>	el <code>setter</code> de <code>TransaccionBase</code>
El archivo <code>.json</code> no existe	<code>FileNotFoundError</code>	<code>cargar_transacciones()</code>
No se puede escribir el archivo	<code>OSError</code>	<code>guardar_transacciones()</code>

Tres decisiones de diseño

Los errores técnicos se traducen a errores del dominio. Un `JSONDecodeError` o un `KeyError` describen lo que le pasó al módulo `json`, no lo que les pasa a los datos. Por eso las funciones los capturan y los vuelven a lanzar como `ValueError` con un mensaje útil: `"al registro le faltan las claves: monto"`. Quien corrige los datos necesita saber qué dato está mal, no en qué línea interna de una librería ocurrió el fallo.

Las claves se validan antes de usarlas. En lugar de dejar que Python lance un `KeyError` seco, la función revisa primero qué claves faltan y las nombra todas juntas. Un solo mensaje resuelve el problema completo, en vez de obligar a corregir un campo por ejecución.

No todos los fallos se tratan igual. Los de un registro suelto se aíslan y la carga continúa, porque los demás registros son válidos y no tienen por qué perderse. Los que impiden continuar —que el archivo no exista, o que su contenido entero no sea JSON— sí detienen el proceso, porque ahí no hay nada que recuperar. Si también se aislaran, el programa seguiría corriendo sobre una lista vacía y el problema aparecería más adelante, peor explicado.

El blindaje del monto y la comprobación

```

Visual Studio Code – serializacion_json.py – encapsulamiento del monto

96     # El monto conserva el encapsulamiento de la Semana 3: el valor real vive en
97     # _monto y solo entra al objeto si supera la validacion del setter. Gracias a
98     # esto, un JSON con un monto de texto o negativo se rechaza al reconstruir el
99     # objeto, no despues.
100    @property
101    def monto(self):
102        """Devuelve el monto protegido de la transaccion."""
103        return self._monto
104
105    @monto.setter
106    def monto(self, nuevo_monto):
107        """Valida y almacena el monto. Lanza ValueError si el dato no sirve."""
108        # float() lanza ValueError si el dato es un texto no numerico.
109        nuevo_monto = float(nuevo_monto)
110
111        if nuevo_monto < 0:
112            raise ValueError("el monto no puede ser negativo")
113
114        self._monto = nuevo_monto
115
116    def calcular_impacto(self):
117        """Cada clase hija define su formula. Este metodo NO viaja en el JSON."""
118        raise NotImplementedError("Cada tipo debe calcular su propio impacto.")
119
120    def __str__(self):
121        return (f"Transaccion [{self.tipo:<8}] - ID: {self.cliente_id} | "
122                f"Monto: ${self.monto:>12,.2f}")

```

Figura 7. El setter de la Semana 3 se conserva: un monto de texto o negativo se rechaza al reconstruir el objeto.

```

Terminal – comprobación con seis entradas defectuosas

--- Manejo de casos de error ---

JSON mal formado      -> ValueError: el texto recibido no es un JSON valido: Expecting property name enclosed in double
quotes: line 1 column 42 (char 41)
Falta la clave monto  -> ValueError: al registro le faltan las claves: monto
Tipo desconocido      -> ValueError: tipo de transaccion desconocido: BONO
Monto no numerico     -> ValueError: could not convert string to float: 'mil'
Monto negativo        -> ValueError: el monto no puede ser negativo
Archivo inexistente   -> FileNotFoundError: no existe el archivo archivo_que_no_existe.json; se genera al ejecutar
guardar_transacciones()

```

Figura 8. Los seis fallos se detectan y se informan; ninguno detiene la ejecución.

Conclusiones y referencias

Lo que aporta la serialización al proyecto y de dónde proviene el sustento conceptual.

Conclusiones

La serialización conecta dos mundos que no se entienden entre sí: el de los objetos, que solo existe mientras el programa corre, y el del almacenamiento y las redes, que únicamente transporta texto. Sin esa traducción, el trabajo de las semanas anteriores solo existiría mientras la ventana estuviera abierta.

La elección de JSON se puede sustentar con razones concretas y no solo con la costumbre: conserva los tipos de dato, explica el significado de cada valor, avisa cuando está mal formado y lo entiende cualquier sistema sin necesidad de un lector propio. En los cuatro puntos, un formato plano queda por debajo.

La traducción no funciona igual en los dos sentidos. Convertir un objeto en texto es directo, pero reconstruirlo exige que el destino conozca la clase, porque el texto solo lleva los datos. Esa asimetría es la razón por la que la fábrica `crear_transaccion()` resulta necesaria en el camino de vuelta y no en el de ida.

El manejo de errores terminó siendo parte del proceso y no un añadido. Un dato que llega de un archivo o de otro sistema no está bajo el control del programa, así que conviene revisarlo antes de usarlo. Validarlo en el momento de reconstruir el objeto, y no más adelante, es lo que impide que un registro corrupto se meta en un cálculo.

Referencias

Cuevas, A. (2016). *Python 3. Curso práctico*. RA-MA Editorial.

Envato Tuts+. (2022). *Serialización y deserialización de objetos Python (Parte 1)*.

Fundación Universitaria CEIPA. (2026). *Fundamentos de Software — Semana 4: Gestión de errores y excepciones*.
<https://fundamentos-de-software.onrender.com/semanas/4>

Python Software Foundation. (2026). *json — JSON encoder and decoder*. Documentación oficial de Python 3.

ENTREGA

Carpeta `ProyectosCeipa/Semana_4_Act_2` · Fundamentos de Software · grupo FUNSO_2_N_H · Deibis Zuluaga Baena · 13 de septiembre de 2026.